ALUMNICONNECT / DBMS PROJECT DESIGN DOCUMENT

Transaction and Consistency Design

ACID boundaries • InnoDB locking • concurrency control • transactional outbox • cross-database consistency

A practical transaction architecture for the Alumni–Student Mentorship and Mock Interview Platform.

DOCUMENT CONTROL	VALUE
Project	AlumniConnect - Alumni–Student Mentorship Platform
Document	Transaction and Consistency Design
Version / date	1.0 / 02 November 2026
Backend baseline	FastAPI modular monolith
Authoritative database	MySQL 8.x with InnoDB
Supporting stores	Redis cache / temporary state; Neo4j recommendation projection
Scope	ACID boundaries, locks, isolation, outbox, retries and concurrency validation

CENTRAL DESIGN RULE
Every durable business transition and its outbox event commit in one MySQL transaction. Redis, Neo4j and external notification providers are updated only after commit and must never decide whether a booking or allocation is valid.

Prepared using the supplied AlumniConnect DBMS Project Blueprint and the previously provided Product Requirements Document, Application Architecture, Workflow and State Diagrams, Business Rules Catalogue, Conceptual ER Model, Use Case Model, Neo4j Graph Design and Redis Design Specification.

Document control, scope and reading guide

This document turns the project’s existing business workflows into explicit transaction boundaries and consistency rules. It is an implementation-oriented specification, not a replacement for the final physical MySQL schema or API contract.

Document conventions

LABEL	INTERPRETATION
[B] Blueprint / requirement	Directly supported by the supplied project documents or PRD.
[R] Recommended implementation	A concrete engineering choice proposed to make the design safe and testable.
[C] Configurable decision	A policy value the team must confirm, such as cancellation cutoff, retry count or timeout.

What this document covers

SECTION	SUBJECT
01–03	Ownership boundaries, transaction concepts and ACID applied to AlumniConnect.
04–07	Transaction catalogue for requests, allocation, availability, booking, cancellation, sessions and feedback.
08–11	InnoDB lock design, isolation levels, anomaly prevention, constraints and indexes.
12–15	Transactional outbox, worker claiming, idempotency, retries and cross-store consistency.
16–20	FastAPI responsibilities, failure handling, concurrency test plan, operational observability and rollout.
Appendices	SQL templates, acceptance evidence, decisions to confirm and source traceability.

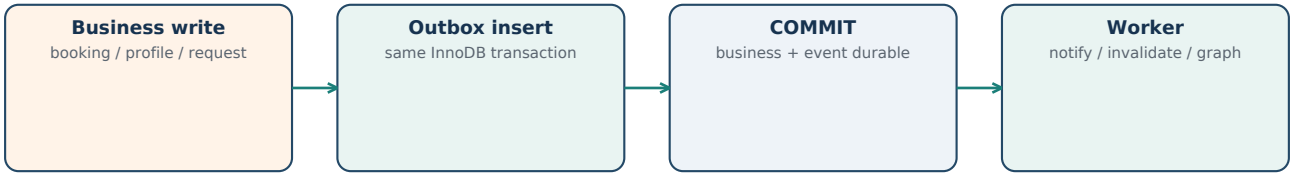
IMPORTANT INTERPRETATION
Where two source documents describe the same principle at different levels, this design preserves the shared invariant: MySQL owns the durable business result; secondary systems are retryable projections. Exact physical table/column names must be aligned with the final relational design.

Source material consulted

The core transaction rules are drawn from the Business Rules Catalogue (Section 12, pp. 24–25; verification catalogue, pp. 28–29), Workflow and State Diagrams (transaction integration, p. 21; procedures and constraints, pp. 18–20), Application Architecture (outbox, p. 11; booking concurrency, pp. 12–13), PRD (FR-20 and FR-23, p. 7; business rules and consistency, pp. 8–10), Conceptual ER Model, Blueprint and Redis Design Specification. The use case model supplies actor and permission boundaries.

1. Executive summary and consistency model

AlumniConnect is a modular monolith backed by polyglot persistence. FastAPI is the single logical application boundary. MySQL 8.x with InnoDB is the system of record for identity, profiles, skills, requests, allocations, availability, bookings, sessions, feedback, reviews, notifications and audit history. Redis accelerates selected reads and stores temporary state. Neo4j supports relationship-oriented mentor discovery.



If SQL rolls back, no event exists. If a worker fails, the committed event remains retryable.

1.1 Ownership by component

COMPONENT	OWNS / DECIDES	MUST NOT DO
FastAPI modules	Authentication, authorization, input validation, orchestration, domain error mapping.	Expose database credentials to the browser or skip business checks.
MySQL / InnoDB	Canonical rows, constraints, business state transitions, transactions, locks, audit and outbox records.	Wait on external notification/graph/cache calls while holding locks.
Redis	Rebuildable display cache, short-lived challenge, rate-limit and idempotency helpers.	Authorize a user, confirm a slot, or own allocation/booking state.
Neo4j	Relationship projection and recommendation paths.	Create authoritative allocations or approve current capacity.
Outbox worker	Post-commit notification delivery, graph projection and cache invalidation.	Process uncommitted work or silently discard failed events.

1.2 Consistency model

• Strong transactional consistency: all business invariants that must hold immediately are enforced in MySQL transactions and constraints.

• Eventual projection consistency: Redis invalidation and Neo4j updates may lag after commit, but they are retryable and do not override MySQL.

• Read freshness: cached display data may have bounded staleness; booking and allocation decisions always re-read and validate current MySQL rows.

NON-NEGOTIABLE
A cache hit, graph path, notification status or front-end calendar state is never proof that a business transition is allowed. The final validation occurs in the authoritative MySQL transaction.

2. Transaction model and transaction boundaries

A transaction is a logical unit of database work. Its operations are treated as one all-or-nothing change. For AlumniConnect, transaction boundaries are defined around business actions, not around individual SQL statements or HTTP requests.

2.1 What belongs inside one transaction

A state-changing operation should include all authoritative writes needed to make the business action true, including the related outbox row and audit entry when those are part of the operation. If any required write or validation fails, the transaction rolls back.

INCLUDE IN THE SAME MYSQL TRANSACTION	DO NOT INCLUDE IN THE MYSQL TRANSACTION
Business row updates and inserts.	Sending email, SMS, push notification or invoking a remote provider.
State/status transition and corresponding history/audit write.	Redis DEL / INCR or cache population.
Required allocation, booking, availability or session changes.	Neo4j node/relationship projection.
Outbox event describing committed side effects.	Long-running report generation or unrelated API calls.
Database uniqueness / FK / check enforcement.	User think time, UI confirmation or network waits.

2.2 Transaction lifecycle

```
START TRANSACTION;
-- Lock and re-read authoritative rows.
-- Validate actor, current state, eligibility and invariants.
-- Apply all required business writes.
-- Insert audit/outbox records where applicable.
COMMIT;

-- On any SQL exception or failed business guard:
ROLLBACK;
```

2.3 Do not split one business action into multiple commits

For example, creating a booking in one transaction and marking the availability slot BOOKED in a second transaction creates a failure window. If the application crashes between commits, the booking and slot can disagree. The booking row, slot transition, planned session record (where the chosen workflow creates it immediately) and outbox event must be committed together.

> DESIGN TEST
> For each business action, ask: “What exact durable state must exist if this action is reported successful?” All of that state belongs in one transaction.

3. ACID properties applied to AlumniConnect

PROPERTY	MEANING	ALUMNICONNECT EXAMPLE / PROOF
Atomicity	All required writes commit together or none do.	Booking insert, slot status change, planned session creation (if used) and BOOKING_CONFIRMED outbox row succeed together or all roll back.
Consistency	A committed transaction preserves database constraints and business invariants.	A booking references an active allocation and the mentor who owns the slot; one request creates at most one allocation; one slot has at most one active booking.
Isolation	Concurrent operations do not create invalid intermediate or conflicting outcomes.	Two students racing for one slot cannot both confirm it; overlapping appointments for the same student or mentor are prevented.
Durability	A committed change survives database/server failure according to configured durability.	A committed request acceptance or booking remains present after a database restart; the outbox event remains available for retry.

3.1 Atomicity - rollback examples

• Allocation insert fails after request acceptance was attempted: request must remain PENDING; no acceptance notification event exists.

• Booking insert succeeds but slot update fails or affects zero rows: roll back the booking and outbox event.

• Session outcome update succeeds but outbox insert fails: roll back the outcome update too.

3.2 Consistency - invariants that must remain true

INVARIANT	DATABASE / PROCEDURE CONTROL
One allocation per accepted request	UNIQUE(mentorship_request_id), request-state guard and capacity validation.
One active booking per slot	Availability row lock plus unique active-slot constraint.
No overlapping confirmed appointments	Student and mentor serialization locks plus overlap query and supporting indexes.
Valid booking ownership	Foreign keys plus procedure checks for allocation, student, mentor and slot owner.
Review only for completed session	Session-state validation plus UNIQUE(session_id, student_id) for mentor review.
No false notification after rollback	Outbox event inserted in the same transaction as the business transition.

3.3 Isolation and durability

Isolation is delivered through the configured InnoDB isolation level together with explicit locking reads and unique constraints. Durability is a database configuration and operational property; the team should verify the deployed InnoDB durability settings and demonstrate persistence after restart rather than claiming durability based only on an API success response.

4. Transaction catalogue: business boundaries

The following matrix is the recommended transaction contract. The list preserves the workflow document's distinction between request, allocation, booking and actual session.

BUSINESS OPERATION	LOCK / VALIDATE	ONE COMMIT MUST INCLUDE
Submit mentorship request	Student identity/profile; target mentor visibility/status; duplicate pending request rule.	Insert PENDING request; insert request-submitted outbox event; optional audit event.
Accept request	Lock request; validate addressed mentor and PENDING state; lock mentor capacity guard; check capacity/duplicate allocation.	Insert one allocation; update request to ACCEPTED; insert outbox notification and audit history.
Reject / cancel request	Lock request; validate actor ownership and PENDING state.	Update status and decision metadata; insert corresponding outbox event; retain original request row.
Publish availability	Lock mentor profile as serialization point; validate active mentor, slot times and no active overlap.	Insert or update availability slots; insert availability-changed outbox event if cache preview is used.
Create booking	Lock student and mentor guards, allocation and slot; re-read status; check overlap and idempotency.	Insert CONFIRMED booking; set slot BOOKED; create SCHEDULED session if required by chosen workflow; insert outbox event.
Cancel booking	Lock participant guards and booking/slot in fixed order; validate actor, current status and cutoff policy.	Mark booking CANCELLED; release slot only if policy permits; insert cancellation outbox event.
Record session outcome	Lock session; validate assigned mentor and SCHEDULED state.	Write COMPLETED / CANCELLED / NO_SHOW outcome and notes; insert session outcome outbox event; audit.
Submit feedback / review	Lock/validate session; require COMPLETED and actual participant; enforce one review per student/session.	Insert feedback or mentor review; insert event if notification is required; preserve prior history.

4.1 Separate worker transaction

Outbox dispatch is not part of the business transaction. The business transaction commits the event intent. A worker later claims and processes the event in a separate short transaction, performs the external side effect outside the lock-holding transaction, and records delivery status separately.

SOURCE ALIGNMENT
Workflow and State Diagrams, Section 17 (p. 21), explicitly lists the transaction contents for accept, create booking, cancel booking, record outcome and submit feedback. The table above turns that catalogue into a single implementation register.

5. Request submission and acceptance transactions

5.1 Submit mentorship request

A new request is a proposal, not an accepted mentorship relationship. It starts in PENDING. The student can submit the request only through the authenticated backend, and the target mentor must be discoverable according to current MySQL profile and visibility rules.

Illustrative SQL - align table and column names to final schema

```
START TRANSACTION;

-- Validate current student and target mentor using authoritative rows.
-- Optionally lock student/mentor guard rows if duplicate-request
-- prevention must serialize concurrent submissions.

INSERT INTO mentorship_request
(student_id, mentor_id, domain_id, objective, status, created_at)
VALUES
(:student_id, :mentor_id, :domain_id, :objective, 'PENDING', NOW());

INSERT INTO notification_outbox
(event_id, event_type, aggregate_type, aggregate_id, status)
VALUES
(:event_id, 'REQUEST_SUBMITTED', 'MentorshipRequest',
LAST_INSERT_ID(), 'PENDING');

COMMIT;
```

5.2 Accept mentorship request

Acceptance is a high-value transaction because it converts a pending request into an allocation. The request row serializes competing accept/reject/cancel operations. The mentor profile row is a guard for capacity checks so concurrent acceptances for different requests do not overrun the mentor’s active allocation limit.

Illustrative SQL - request and mentor guard lock

```
START TRANSACTION;

SELECT student_id, mentor_id, status
FROM mentorship_request
WHERE request_id = :request_id
FOR UPDATE;

-- Verify caller is the addressed mentor. Require status = PENDING.
SELECT mentor_id, account_status, max_active_mentees
FROM mentor_profile
WHERE mentor_id = :mentor_id
FOR UPDATE;

-- Count active allocations after the capacity row is locked.
-- Reject if mentor is inactive or at configured capacity.

INSERT INTO mentor_allocation
(request_id, student_id, mentor_id, status, created_at)
VALUES (:request_id, :student_id, :mentor_id, 'ACTIVE', NOW());

UPDATE mentorship_request
SET status = 'ACCEPTED', decided_at = NOW()
WHERE request_id = :request_id AND status = 'PENDING';
-- Require ROW_COUNT() = 1.

INSERT INTO notification_outbox (..., event_type, ...)
VALUES (..., 'REQUEST_ACCEPTED', ...);

COMMIT;
```

5.3 Acceptance race

Two concurrent acceptance attempts for the same request must not create two allocations. One transaction locks the request, validates PENDING, inserts the allocation and updates the request. The second waits; after lock acquisition it re-reads and sees ACCEPTED, then returns a conflict/already-processed result. A UNIQUE constraint on allocation.request_id is the final database backstop.

REQUIRED RESULT
Exactly one ACTIVE allocation; request status ACCEPTED; one committed outbox intent. The competing attempt must not create a second allocation or notification.

6. Reject, cancel request and publish availability

6.1 Reject or cancel a pending request

Reject and cancel are separate actor-authorized transitions from PENDING. The addressed mentor rejects; the student who opened the request cancels. Both operations lock the request and re-check the status inside the transaction. Rejected and cancelled rows remain in history.

Request rejection/cancellation template

```
START TRANSACTION;
SELECT student_id, mentor_id, status
FROM mentorship_request
WHERE request_id = :request_id
FOR UPDATE;

-- Check actor ownership and require current status = PENDING.
UPDATE mentorship_request
SET status = :new_status, decided_at = NOW(), decision_reason = :reason
WHERE request_id = :request_id AND status = 'PENDING';
-- Require ROW_COUNT() = 1.
INSERT INTO notification_outbox (..., event_type, ...)
VALUES (..., :request_decision_event, ...);
COMMIT;
```

6.2 Publish mentor availability

Source terminology note: the Business Rules Catalogue describes the initial slot state as OPEN, while one Workflow SQL example uses AVAILABLE. The physical schema must choose one canonical status value. Code examples in this specification use OPEN. Availability publication should be serialized per mentor. Lock the mentor profile row before checking for overlapping active slots and inserting the new slot(s). This parent-row lock prevents two simultaneous publication calls from each observing “no overlap” and inserting conflicting time ranges.

- Validate start_at < end_at and interpret/store time instants consistently (the catalogue recommends UTC).
- Check mentor account/profile is active and permitted to publish slots.
- Check no overlapping active availability exists unless group/parallel sessions are explicitly supported.
- Do not delete historical slots that have bookings; block, expire or otherwise preserve history.

6.3 Overlap predicate

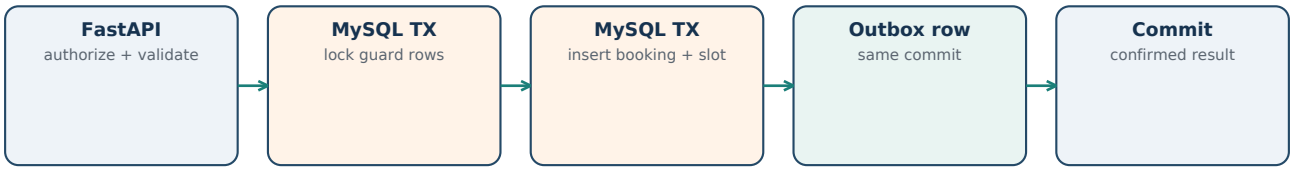
The business-rules catalogue uses half-open time intervals [start_at, end_at). Two appointments overlap when existing.start_at is earlier than the requested end and existing.end_at is later than the requested start. Thus 10:00–10:30 and 10:30–11:00 are adjacent, not overlapping.

Half-open interval overlap test

```
SELECT COUNT(*)
FROM mentor_availability a
WHERE a.mentor_id = :mentor_id
AND a.status IN ('OPEN','BOOKED')
AND a.start_at < :requested_end
AND a.end_at > :requested_start;
```

7. Booking transaction - critical consistency path

Booking is the most critical transaction in AlumniConnect. Two students may try to reserve one mentor slot at nearly the same time. A calendar preview is only a read hint. MySQL must decide the result by locking and validating current rows inside the booking transaction.



Redis / Neo4j / email are not called while the InnoDB locks are held.

7.1 Preconditions

- The actor is the student associated with the active allocation.
- The allocation is ACTIVE and the selected slot belongs to the same mentor.
- The slot exists, is OPEN/AVAILABLE and starts in the future.
- The student and mentor have no overlapping CONFIRMED appointment.
- Mentor capacity and any domain/business-specific booking policy are satisfied.
- The request is not a duplicate retry; idempotency key and request fingerprint are consistent.

7.2 Fixed lock order



Use the same order in every procedure that touches these resources.

The Business Rules Catalogue recommends a consistent order such as student profile → mentor profile → allocation → availability slot → booking/session. All procedures that touch these shared resources must use the same order. If a procedure starts from booking_id, it may perform a non-locking lookup to discover the involved IDs, then acquire locks in the canonical order and re-read the booking row before mutating it.

7.3 Why lock the participant profile rows?

Locking only the availability slot serializes two users competing for the exact same slot. It does not prevent one student from booking two overlapping slots with two different mentors, nor does it prevent one mentor's separate slots from being booked at overlapping times. The student and mentor profile rows act as serialization points for all booking operations involving those participants.

8. Booking transaction - SQL implementation template

The following is pseudocode-style SQL to show the ordering and invariant checks. It is not a drop-in stored procedure; adapt names, exception handlers and result codes to the final relational schema.

Booking transaction skeleton

```
START TRANSACTION;

-- 1. Lock student and mentor guard rows in the canonical order.
SELECT student_id FROM student_profile
WHERE student_id = :student_id FOR UPDATE;
SELECT mentor_id, account_status, max_active_mentees
FROM mentor_profile WHERE mentor_id = :mentor_id FOR UPDATE;

-- 2. Lock allocation and availability, then re-read their states.
SELECT allocation_id, status, student_id, mentor_id
FROM mentor_allocation
WHERE allocation_id = :allocation_id FOR UPDATE;
SELECT availability_id, mentor_id, status, start_at, end_at
FROM mentor_availability
WHERE availability_id = :slot_id FOR UPDATE;

-- 3. Validate actor, allocation, owner, future time, OPEN status.
-- 4. Check overlaps for both student and mentor using [start,end).
-- 5. Check capacity and idempotency.

INSERT INTO booking (...) VALUES (..., 'CONFIRMED', ...);
UPDATE mentor_availability
SET status = 'BOOKED'
WHERE availability_id = :slot_id AND status = 'OPEN';
-- Require ROW_COUNT() = 1 or raise an error.

-- Create SCHEDULED session here if that is the selected lifecycle rule.
INSERT INTO notification_outbox (..., event_type, ...)
VALUES (..., 'BOOKING_CONFIRMED', ...);

COMMIT;
-- On any exception: ROLLBACK and rethrow mapped error.
```

8.1 Database backstops

RULE	RECOMMENDED ENFORCEMENT
Availability slot consumed at most once	UNIQUE(availability_id) if a slot can never be reused; otherwise unique nullable active_slot_id generated for active bookings.
No duplicate accepted allocation	UNIQUE(mentorship_request_id) in mentor_allocation.
No duplicate client booking effect	Durable UNIQUE(actor_id, idempotency_key) in MySQL; Redis idempotency key is supplementary.
Valid references	Foreign keys from booking to student, mentor, availability and (where required) allocation.
Valid time range	CHECK(end_at > start_at), with API validation and consistent time-zone handling.
Fast conflict lookup	Indexes on booking(student_id, status, start_at, end_at) and booking(mentor_id, status, start_at, end_at).

8.2 Generated active-slot key pattern (optional schema technique)

Unique active booking while retaining cancelled history

```
-- Illustrative MySQL pattern; validate support/version and schema design.
active_slot_id BIGINT GENERATED ALWAYS AS (
CASE WHEN booking_status = 'CONFIRMED' THEN availability_id
ELSE NULL END
) STORED,
UNIQUE KEY uq_active_slot (active_slot_id)
```

9. Cancel and reschedule booking safely

9.1 Cancellation is a state transition, not a delete

The original booking remains for history. The procedure must lock the booking and the resources needed to avoid concurrent cancellation/booking races, authorize one of the participants (or an explicitly authorized administrator), require current status CONFIRMED and apply the agreed cancellation cutoff.

Booking cancellation template

```
START TRANSACTION;

-- Discovery read only: obtain IDs; do not lock booking yet.
SELECT student_id, mentor_id, allocation_id, availability_id
FROM booking WHERE booking_id = :booking_id;

-- Lock guards in the canonical order: student -> mentor ->
-- allocation -> availability slot -> booking. Re-read status.
SELECT student_id FROM student_profile
WHERE student_id = :student_id FOR UPDATE;
SELECT mentor_id FROM mentor_profile
WHERE mentor_id = :mentor_id FOR UPDATE;
SELECT allocation_id, status FROM mentor_allocation
WHERE allocation_id = :allocation_id FOR UPDATE;
SELECT availability_id, status FROM mentor_availability
WHERE availability_id = :slot_id FOR UPDATE;
SELECT booking_status FROM booking
WHERE booking_id = :booking_id FOR UPDATE;

-- Verify actor, require CONFIRMED, apply cancellation policy.
UPDATE booking
SET booking_status = 'CANCELLED', cancelled_at = NOW(),
cancellation_reason = :reason
WHERE booking_id = :booking_id
AND booking_status = 'CONFIRMED';

-- Re-open slot only if cancellation policy allows it.
UPDATE mentor_availability SET status = 'OPEN'
WHERE availability_id = :slot_id AND status = 'BOOKED';

INSERT INTO notification_outbox (..., event_type, ...)
VALUES (..., 'BOOKING_CANCELLED', ...);
COMMIT;
```

9.2 Cancellation policy decisions

The supplied documents do not set a precise cancellation cutoff or guarantee that every cancellation reopens the slot. Those are configurable product decisions. The database procedure must apply the final policy consistently and preserve the original booking and its audit history.

• Whether a slot is reopened depends on timing and the approved cancellation rule.

• A confirmed booking cannot be edited in place to a different participant or slot.

• Rescheduling should create a replacement booking linked to the original; the original becomes CANCELLED/RESCHEDULED according to the agreed state model.

9.3 Reschedule as one atomic operation

The safest approach is to reserve the new slot and transition the old booking in one transaction. Lock both participant rows, both availability rows in ascending stable ID order, then the original booking. If the new slot cannot be reserved, roll back so the original booking remains confirmed. Preserve a rescheduled_from_booking_id link for auditability.

SOURCE-DEFINED INVARIANT
The business rules catalogue says the original booking history must be preserved; rescheduling should not silently overwrite participant, slot or timestamp fields.

10. Session outcome, feedback and mentor review

10.1 Record session outcome

A booking is a planned appointment. A mentorship session is the actual meeting and its outcome. The assigned mentor records the result after the offline meeting. The session state transition, outcome details, audit history and event intent must commit together.

Session outcome transaction

```
START TRANSACTION;
SELECT booking_id, mentor_id, session_status
FROM mentorship_session
WHERE session_id = :session_id FOR UPDATE;

-- Verify assigned mentor (or audited administrator override).
-- Require session_status = 'SCHEDULED'.
-- Validate requested outcome in ('COMPLETED','CANCELLED','NO_SHOW').
UPDATE mentorship_session
SET session_status = :outcome,
completed_at = CASE WHEN :outcome = 'COMPLETED'
THEN NOW() ELSE NULL END,
discussion_topics = :topics,
performance_observations = :observations,
next_steps = :next_steps
WHERE session_id = :session_id;

INSERT INTO notification_outbox (..., event_type, ...)
VALUES (..., :session_event, ...);
COMMIT;
```

10.2 Submit session feedback

• Lock or otherwise consistently validate the session state; require COMPLETED.

• Resolve participants through the session → booking relationship; compare the authenticated actor to the participant record.

• Insert feedback in a separate durable row; preserve author, session and timestamp.

• Insert a notification event in the same transaction only if the workflow requires it.

10.3 Submit mentor review

A mentor review is distinct from general session feedback. Only the student participant may review the mentor, and only after a completed session. A unique key such as (session_id, student_id) prevents duplicate review submissions for the same session. Do not overwrite an existing review silently.

Mentor review transaction

```
START TRANSACTION;
SELECT session_status, student_id, mentor_id
FROM v_session_participants
WHERE session_id = :session_id FOR UPDATE;

-- Require COMPLETED and reviewer = student_id.
INSERT INTO mentor_review
(session_id, student_id, mentor_id, rating, review_text, created_at)
VALUES (:session_id, :student_id, :mentor_id, :rating,
:review_text, NOW());
-- UNIQUE(session_id, student_id) protects duplicate review.
INSERT INTO notification_outbox (..., event_type, ...)
VALUES (..., 'FEEDBACK_SUBMITTED', ...);
COMMIT;
```

11. InnoDB locking strategy

Use row-level locks for the records that represent resources being changed or whose state/capacity must remain stable while the transaction decides. Use SELECT ... FOR UPDATE for the critical decision rows.

11.1 Canonical lock order



Use the same order in every procedure that touches these resources.

ORDER	RESOURCE	WHY IT IS LOCKED
1	Student profile	Serializes the student-side overlapping appointment check.
2	Mentor profile	Serializes mentor-side overlap and capacity/availability checks.
3	Allocation	Re-checks active student-mentor relationship and its status.
4	Availability slot	Serializes attempts to claim the same slot and re-checks OPEN state.
5	Booking / session	Locks the specific lifecycle record being changed.

11.2 Locking rules

• Use the same order in create, cancel and reschedule procedures. Sort multiple rows of the same type by immutable ID before locking them.

• Re-read state after acquiring the lock. A value read before locking is only a discovery hint.

• Keep transactions short. Do not call Redis, Neo4j, email providers or remote APIs while holding database locks.

• Use indexes that make locking reads selective; an unsuitable range scan can lock more records than intended.

• Use constraints as backstops even when procedures lock rows correctly.

11.3 Two-phase locking and deadlocks

The database may hold locks until commit/rollback. Two-phase locking can provide conflict-serializable behaviour, but it does not eliminate deadlocks. Deadlocks can occur when two transactions wait in a cycle, especially when they acquire shared resources in different orders. InnoDB detects a deadlock and rolls back a victim; the application must retry the entire transaction when it is safe.

12. Concurrency anomalies and controls

ANOMALY / RACE	ALUMNICONNECT EXAMPLE	CONTROL
Dirty read	A user sees a booking that another transaction later rolls back.	Do not use READ UNCOMMITTED for business data; use committed reads.
Lost update	Two operations update the same slot or booking state and one overwrites the other.	Lock target row with FOR UPDATE; conditional UPDATE; check ROW_COUNT().
Non-repeatable read	A workflow reads allocation ACTIVE, then another transaction ends it before booking.	Use locking re-read for the write decision; validate allocation inside the transaction.
Phantom / overlap race	Two transactions each see no overlapping booking and insert different overlapping appointments.	Lock student and mentor guard rows before overlap query; same lock protocol for all writers.
Double-booking same slot	Two students claim the same availability row.	Lock availability row; unique active-slot constraint; one transaction wins.
Duplicate HTTP submission	Client retries after timeout and creates another booking.	Idempotency key + durable MySQL unique constraint + return original result.
Capacity write skew	Two requests are accepted after each independently counts one free mentor slot.	Lock mentor profile capacity row before count/insert; unique request allocation constraint.

12.1 Why overlap checking needs a parent-row lock

A normal overlap query can return zero rows to two concurrent transactions. If they each insert a conflicting appointment, a check-then-insert race has occurred. Locking only existing booking rows is insufficient when the set is empty. The design therefore locks a stable parent row (student_profile and mentor_profile) as the serialization point, and every booking writer follows that convention.

12.2 Half-open intervals

Time overlap predicate for [start_at, end_at)

```
existing.start_at < requested.end_at
AND existing.end_at > requested.start_at
```

This definition allows adjacent sessions. A session ending exactly when another begins does not overlap. Use UTC instants (as recommended by the catalogue) and convert for display in the participant’s local time zone.

CONCURRENCY PROOF
Run tests with two independent database connections and a synchronization barrier. Sequential API calls do not prove the design is safe under concurrent execution.

13. Isolation levels and read strategy

The source business-rules catalogue notes that InnoDB commonly uses REPEATABLE READ by default, but requires the team to confirm the actual server setting. It also notes that READ COMMITTED may be appropriate for some application reads, and SERIALIZABLE should not be enabled globally merely to solve one booking race.

LEVEL	PRACTICAL INTERPRETATION	ALUMNICONNECT GUIDANCE
READ UNCOMMITTED	May expose uncommitted changes (dirty reads).	Do not use for business state, booking, allocation or session decisions.
READ COMMITTED	Each consistent read sees committed data as of that statement.	May be suitable for selected ordinary reads; still use explicit locking for resource reservation and capacity decisions.
REPEATABLE READ	Consistent reads in a transaction use a stable snapshot; locking/range reads have InnoDB-specific locking behaviour.	Reasonable initial deployment baseline if it is the verified server default; use FOR UPDATE and constraints for critical decisions.
SERIALIZABLE	Strongest isolation; may add broad blocking and contention.	Do not enable globally as a substitute for correct lock design. Use only after workload-specific analysis.

13.1 Recommended baseline

• Verify the actual MySQL global and session transaction isolation variables during deployment.

• Start with the verified InnoDB default (commonly REPEATABLE READ) unless testing shows a justified reason to configure otherwise.

• Use locking reads for booking, acceptance, cancellation and state transitions; do not rely on an earlier plain SELECT as the decision point.

• Use ordinary committed reads for directory/profile/report reads, with cache freshness and privacy handled by the application.

• Test the concurrency scenarios at the actual configured isolation level and with the final index set.

13.2 Inspect current configuration

Configuration inspection / optional session setting

```
SELECT @@transaction_isolation;
SELECT @@global.transaction_isolation;

-- Set for a specific connection before starting a transaction,
-- only if this is an approved deployment decision:
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

DO NOT ASSUME
Do not write “the application uses REPEATABLE READ” in the final report until the deployed MySQL instance has been checked. Record the observed setting in the test evidence.

14. Constraints, indexes and database enforcement

Stored procedures and backend services orchestrate multi-row workflows. Constraints provide the final defence against invalid database states, including unexpected direct writes and concurrency races.

OBJECT	RULE TO ENFORCE	PURPOSE
FK booking → availability	Booking references an existing slot.	Prevents orphaned bookings.
FK booking → allocation	Booking belongs to a valid allocation (where required).	Connects reservation to the accepted mentorship relationship.
UNIQUE allocation.request_id	At most one allocation per mentorship request.	Prevents duplicate acceptance effects.
UNIQUE active slot key	At most one active booking consumes an availability slot.	Backstop for double-booking.
UNIQUE actor + idempotency key	One durable operation per actor/key.	Protects retries even if Redis is flushed.
UNIQUE review session + student	One mentor review per student per completed session.	Protects review uniqueness.
CHECK end_at > start_at	Time ranges have positive duration.	Rejects invalid appointment windows.
CHECK status domain	Status values restricted to valid lifecycle states.	Prevents invalid state strings.

14.1 Supporting indexes

TABLE / INDEX	KEY ORDER (ILLUSTRATIVE)	USED BY
mentorship_request	(mentor_id, status, created_at)	Mentor inbox and pending request list.
mentor_allocation	UNIQUE(request_id), plus (mentor_id, status)	One allocation per request and mentor capacity check.
mentor_availability	(mentor_id, status, start_at, end_at)	Availability and range lookup.
booking by student	(student_id, booking_status, start_at, end_at)	Student overlap check and calendar.
booking by mentor	(mentor_id, booking_status, start_at, end_at)	Mentor overlap check and calendar.
notification_outbox	(status, next_attempt_at, created_at)	Efficient pending event polling.
mentor_review	UNIQUE(session_id, student_id)	Duplicate review prevention.

14.2 Conditional uniqueness

A design may need to retain cancelled bookings while allowing the same slot to be booked later. MySQL does not provide a simple partial unique index in the same way as some other database systems. A generated nullable active_slot_id column can be considered: it contains availability_id only for active/CONFIRMED rows and NULL otherwise, with a UNIQUE index. Verify the final status semantics and MySQL version before using this pattern.

15. Stored procedures, triggers and audit design

15.1 Stored procedures own multi-row workflows

Use procedures or a clearly defined FastAPI transaction service for operations such as sp_respond_to_mentorship_request, sp_create_booking, sp_cancel_booking, sp_record_session_outcome and sp_submit_mentor_review. The workflow documents recommend database-side guards and procedures for critical state changes.

15.2 Triggers are guardrails and audit helpers

TRIGGER IDEA	TIMING	RESPONSIBILITY
Request state guard	BEFORE UPDATE mentorship_request	Reject invalid transitions using SIGNAL.
Request audit	AFTER UPDATE mentorship_request	Write old/new state and timestamp when status changes.
Session state guard	BEFORE UPDATE mentorship_session	Allow SCHEDULED → COMPLETED/CANCELLED/NO_SHOW; reject terminal regression.
Session audit	AFTER UPDATE mentorship_session	Record outcome status changes and actor metadata where available.
Booking audit	AFTER INSERT/UPDATE booking	Record confirmation/cancellation changes.
Review validation (optional)	BEFORE INSERT mentor_review	Additional rating/state safeguard; main workflow validation remains in procedure.

15.3 What triggers should not do

• Do not implement the entire multi-table booking workflow inside a trigger.

• Do not send email or call HTTP services from a trigger.

• Do not issue COMMIT, ROLLBACK or START TRANSACTION inside a trigger; it executes inside its caller transaction.

• Do not create the same outbox event in both a trigger and a procedure. Choose one owner for event creation.

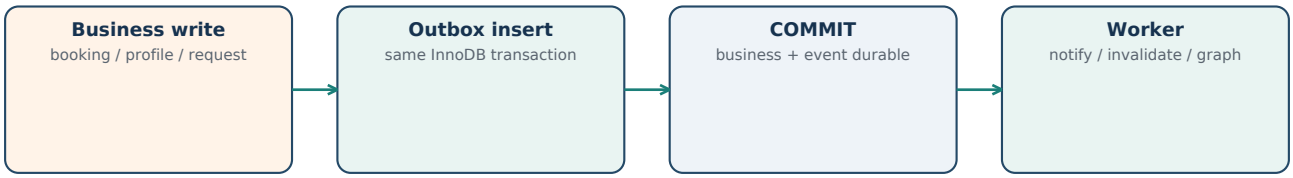
• Do not rely only on application-side validation for a status transition if a database guard is required.

15.4 Audit and outbox ownership

The application/procedure should create exactly one outbox event for each business event. Triggers may create audit rows for state changes if that is the chosen audit strategy. Ensure that both audit and outbox records participate in the original SQL transaction so rollback removes them as well.

16. Transactional outbox pattern

The transactional outbox solves the dual-write problem. FastAPI must not commit a business row and then separately “hope” it can publish a notification or graph/cache update. Instead, the business transaction inserts a durable event intent in MySQL. The worker processes only committed outbox rows.



If SQL rolls back, no event exists. If a worker fails, the committed event remains retryable.

16.1 Required sequence

STAGE	ACTION	CONSISTENCY RESULT
1. Validate	FastAPI authorizes the actor and validates inputs/current domain context.	No secondary store has changed.
2. Begin SQL transaction	Update business rows and insert outbox event in the same InnoDB transaction.	Business update and event are atomic.
3. Commit	Commit once all required writes succeed.	MySQL is the new authoritative state.
4. Worker claim	Worker claims a committed pending event in a short SQL transaction.	Event is retryable and owned by one worker lease.
5. Side effect	Worker sends notification, invalidates Redis or updates Neo4j outside the SQL lock transaction.	Remote slowness does not hold business row locks.
6. Mark processed	Worker updates outbox status, attempt metadata and timestamps.	Delivery/projection outcome is observable.

16.2 Event envelope

Minimal event payload - stable IDs, no unnecessary personal data

```
{
"event_id": "uuid",
"event_type": "MentorSkillsUpdated",
"aggregate_type": "Mentor",
"aggregate_id": 482,
"aggregate_version": 17,
"occurred_at": "2026-11-02T12:00:00Z",
"payload": { "changed_fields": ["skills"] }
}
```

Prefer stable IDs, event type and aggregate version over embedding full profile or session content. The worker can re-read authoritative details from MySQL when necessary.

17. Outbox worker design and delivery semantics

17.1 Claim safely, then release locks

A worker should not hold an InnoDB row lock while waiting for email/SMS or another remote system. In MySQL 8, a worker may claim a small batch using SELECT ... FOR UPDATE SKIP LOCKED (where appropriate), update lease/claimed metadata, and commit. It then performs the side effect outside that transaction.

Worker claim / process / acknowledge outline

```
-- Transaction A: claim pending work (template).
START TRANSACTION;
SELECT event_id
FROM notification_outbox
WHERE status = 'PENDING'
AND next_attempt_at <= NOW()
ORDER BY created_at
LIMIT 20
FOR UPDATE SKIP LOCKED;

-- Update selected rows to PROCESSING with worker_id/lease_until.
-- Commit claim before remote work.
COMMIT;

-- Outside transaction: deliver notification / project graph / invalidate cache.

-- Transaction B: mark SENT/PROCESSED or schedule retry.
START TRANSACTION;
UPDATE notification_outbox
SET status = 'SENT', processed_at = NOW()
WHERE event_id = :event_id AND worker_id = :worker_id;
COMMIT;
```

17.2 Delivery is at-least-once; make consumers idempotent

A worker may crash after a provider accepts a notification but before MySQL is updated to SENT. On restart the event can be retried. Exactly-once delivery across an external provider and MySQL is generally not guaranteed by the outbox alone. Design for at-least-once attempts and deduplicate using provider idempotency support or a stable event/recipient key.

17.3 Retry and dead-letter lifecycle

STATUS	MEANING	NEXT ACTION
PENDING	Committed, not yet claimed.	Eligible for worker claim when due.
PROCESSING	Claimed under a lease.	Worker processes; expired lease can be reclaimed.
SENT / PROCESSED	Side effect acknowledged.	Retain status for audit/operations.
RETRY	Transient error; next_attempt_at is set.	Retry with bounded exponential backoff and jitter.
DEAD_LETTER	Permanent or repeatedly failing event.	Inspect, correct cause and explicitly re-drive if safe.

17.4 Minimum outbox fields

Recommended fields: event_id (UUID/PK), event_type, aggregate_type, aggregate_id, aggregate_version, recipient_id if relevant, payload, status, attempts, next_attempt_at, worker_id, lease_until, created_at, processed_at, last_error and dedupe_key. Use a unique dedupe_key for a stable business event/recipient combination where the domain requires it.

> NO FALSE NOTIFICATION
> If the business transaction rolls back, the outbox row rolls back and the worker sees nothing. If the worker is offline, the business transaction still commits; the event remains pending for later retry.

18. Cross-database consistency: Redis and Neo4j

Redis and Neo4j are updated from committed MySQL events. They do not participate in the MySQL ACID transaction. This is intentional: it prevents a Redis or Neo4j outage from rolling back a legitimate profile, booking or session operation.

EVENT	SECONDARY ACTION	BUSINESS SAFETY
MentorProfileUpdated / MentorSkillsUpdated / MentorDomainsUpdated	Delete mentor public-profile cache; advance mentor/recommendation cache epoch; update Neo4j projection.	MySQL profile/skills remain authoritative.
StudentSkillsUpdated / StudentInterestsUpdated	Advance student/recommendation cache epoch; update student graph relationships.	Recommendations may be briefly stale; MySQL eligibility check remains.
MentorVisibilityOrAccountStatusChanged	Delete public profile cache and invalidate recommendations; update graph projection.	Backend rechecks visibility/status in MySQL before displaying or acting.
MentorAllocationCreated / AllocationEnded	Invalidate mentor eligibility/recommendation cache; update MENTORED_BY relationship projection.	Capacity and active allocation count are checked in MySQL.
AvailabilityChanged / BookingConfirmed / BookingCancelled	Delete relevant mentor/date availability preview.	A preview is never a booking decision.
MentorReviewCreated / FeedbackAggregatesUpdated	Refresh or invalidate ranking cache derived from MySQL.	Rating history is stored in MySQL.

18.1 Redis invalidation idempotency

DEL of a missing key is naturally safe to replay. INCR is not idempotent: a repeated event can advance an epoch more than once. If epoch correctness depends on a single application of the event, use an event marker and invalidation in one Redis Lua script/atomic operation, or use a deterministic version value from the committed aggregate version. Do not write a deduplication marker in a separate command before the invalidation.

18.2 Neo4j projection

Graph updates should use stable MySQL IDs and idempotent MERGE/upsert operations. A failed or lagging projection is a recommendation freshness issue, not a reason to skip MySQL status, privacy, capacity or eligibility checks. Redis and Neo4j may be rebuilt from MySQL/outbox history according to the recovery plan.

18.3 Cache consistency boundary

The Redis Design Specification defines cache-aside, bounded TTL, targeted DEL, cache epochs and event mapping. It also says authorization must run for cache hits and availability preview must never confirm a booking. This document treats those rules as downstream consistency obligations of the outbox worker.

19. Idempotency, duplicate submissions and error handling

19.1 Two layers of idempotency

LAYER	RESPONSIBILITY	LIMITATION
Redis idempotency helper	Fast short-lived record keyed by actor + operation + idempotency key; stores request fingerprint and compact result reference.	May be evicted/restarted; cannot be the only durable duplicate protection.
MySQL idempotency record / UNIQUE key	Durable deduplication for a state-changing operation.	Must be committed atomically with the booking/request effect.
Business UNIQUE constraints	Prevent duplicates such as one allocation per request or one review per session/student.	Do not replace request fingerprint comparison or actor authorization.

19.2 Request replay rules

• Same idempotency key + same request fingerprint + completed operation: return the original result/reference.

• Same key + different request fingerprint: reject as idempotency-key reuse conflict.

• Same key + operation still in progress: return a controlled in-progress response or wait briefly according to API policy.

• Redis missing or unavailable: MySQL unique constraints and transaction rules still prevent duplicate durable effects.

19.3 MySQL error mapping

MYSQL ERROR	INTERPRETATION	APPLICATION ACTION
1213 - Deadlock found	InnoDB selected this transaction as deadlock victim.	Roll back and retry the entire transaction/procedure with small bounded retry count and jitter.
1205 - Lock wait timeout	A lock was not acquired within timeout.	Ensure full transaction rollback; log contention; return controlled retryable failure where appropriate.
1062 - Duplicate key	Could be duplicate slot, allocation, review or idempotency replay.	Inspect constraint name; return a meaningful conflict or original result.
SQLSTATE 45000	Business rule SIGNAL (invalid state, capacity, overlap, etc.).	Do not blindly retry; return domain validation/conflict error.

19.4 Retry policy

Use a small bounded retry count, exponential delay and jitter for known transient lock/deadlock conditions. Retry the entire transaction from the start, not only the failed statement, because locks may have been released and earlier reads may no longer be valid. Do not retry deterministic business-rule failures or malformed input.

RETRY SAFETY
A retried transaction must re-authorize and re-read the current state, and must reuse the same idempotency key. Never replay a stale in-memory decision.

20. Failure handling and graceful degradation

FAILURE	BUSINESS TRANSACTION RESULT	RECOVERY / USER BEHAVIOUR
Redis unavailable during profile update	MySQL profile update + outbox event may still commit.	Outbox remains pending; worker retries invalidation. Ordinary reads can fall back to MySQL.
Neo4j unavailable during mentor update	MySQL update + projection outbox event commits.	Retry graph projection; recommendations may be stale; MySQL validates candidate status.
Email provider unavailable	Business state and notification outbox event remain committed.	Retry with backoff; eventually dead-letter for inspection. Never roll back accepted request/booking.
Crash before MySQL commit	No business change or outbox event is committed.	Client may retry safely using idempotency key.
Crash after commit before API response	Business change and outbox intent are committed.	Client retry returns original durable operation result; no duplicate booking.
Worker crash after remote send before marking SENT	Event may be sent again.	Use provider/event idempotency key; reconcile delivery state.
Deadlock / lock timeout	Current business transaction aborts.	Retry whole transaction only for configured transient errors.
Redis flush / eviction	Cache and temporary helpers may disappear.	Rebuild read caches; durable SQL constraints preserve booking and allocation correctness.

20.1 Redis-specific failure policy

Profile/catalogue cache misses fall back to MySQL. Recommendation cache misses recompute through Neo4j plus MySQL eligibility or return a controlled retry if dependencies are unavailable. Login challenge failure must not bypass verification. Rate limiting must use an application fallback or controlled response. Durable idempotency must still be protected in MySQL.

20.2 Do not compensate by weakening consistency

Do not confirm a booking because a Redis preview says OPEN; do not accept a mentor because a cached capacity count says there is space; do not show private mentor details because they are present in cache; and do not drop outbox events to make a worker appear healthy.

21. FastAPI responsibilities and transaction service design

21.1 Recommended request path

LAYER	RESPONSIBILITY
Router / API endpoint	Parse request, authenticate request context, invoke application service, map outcome to HTTP response.
Application service	Coordinate domain validation and one transaction boundary; decide which repositories must participate.
MySQL repositories / procedures	Execute locking reads, state changes, constraints and outbox inserts.
Outbox worker	Process committed events and secondary effects.
Redis / Neo4j adapters	Provide cache/discovery functionality; never own durable business decision.

21.2 Pseudocode - booking orchestration

Conceptual FastAPI service - exception/retry scaffolding omitted

```
async def create_booking(cmd, actor):
# Authenticate/authorize before entering the unit of work.
idem = await idem_store.begin(actor.id, cmd.key, cmd.fingerprint)
if idem.is_completed:
return idem.original_result

try:
async with mysql.transaction() as tx:
# Lock student -> mentor -> allocation -> slot.
# Re-read statuses and validate overlap/capacity.
booking = await booking_repo.create_confirmed(tx, cmd)
await availability_repo.mark_booked(tx, cmd.slot_id)
await session_repo.create_scheduled_if_required(tx, booking)
await outbox_repo.add(tx, event_for(booking))
# SQL is committed here. No network calls occurred in tx.
await idem_store.complete(actor.id, cmd.key, booking.id)
return booking
except DeadlockOrLockTimeout:
# Retry whole transaction under bounded retry policy.
raise RetryableConflict()
```

21.3 Important engineering notes

- Do not set idempotency to COMPLETED before SQL commit.
- If Redis is unavailable, the MySQL durable idempotency record remains the final defence.
- Avoid holding one connection/transaction open while waiting on Redis, Neo4j or a notification provider.
- Use explicit transaction helpers/context managers so rollback occurs on every exception path.
- Keep routers thin; put reusable workflow logic in application services or stored procedures.

22. Concurrency and transaction verification plan

Test the database boundary using independent MySQL connections. A single sequential test cannot expose the race conditions this design is intended to prevent.

TEST ID	SCENARIO	EXPECTED RESULT
TX-01	Two mentors/clients attempt to accept the same request simultaneously.	One allocation only; request ACCEPTED once; duplicate attempt receives conflict/already processed.
TX-02	Two acceptance attempts for different requests exceed one mentor capacity.	Capacity not exceeded; only permitted number of active allocations commit.
TX-03	Two students book the same availability slot concurrently.	Exactly one CONFIRMED booking; slot BOOKED; losing transaction reports unavailable/conflict.
TX-04	One student books overlapping slots with different mentors concurrently.	At most one confirmed overlapping booking; student parent lock serializes the conflict check.
TX-05	One mentor has overlapping confirmed appointments from different students.	Second booking rejected; mentor parent lock serializes.
TX-06	Adjacent appointments 10:00–10:30 and 10:30–11:00.	Both accepted; half-open interval rule treats them as non-overlapping.
TX-07	Client retries booking with same idempotency key and same fingerprint.	Original booking result returned; no second booking/session.
TX-08	Same idempotency key reused with different payload.	Rejected as key reuse conflict.
TX-09	Inject failure after booking insert but before slot update.	Full rollback; no booking/session/outbox; slot remains OPEN.
TX-10	Outbox worker is offline during booking commit.	Booking commits; event remains PENDING and is delivered later.
TX-11	Worker crashes after provider accepts notification but before SENT update.	Retry occurs; event/recipient idempotency prevents duplicate visible notification where provider supports it.
TX-12	Deadlock 1213 occurs.	Entire transaction rolled back and retried under bounded retry rule.
TX-13	Session feedback submitted while session not COMPLETED.	Rejected; no feedback or outbox row created.
TX-14	Two mentor reviews submitted for one session by same student.	One accepted; duplicate blocked by unique key.

22.1 Same-slot race procedure

Prepare one OPEN slot and two eligible students with ACTIVE allocations. Open two independent database sessions. Use a barrier so both booking operations begin at nearly the same time. Capture lock wait, transaction outcome, row counts and final table state. Verify one booking, one scheduled session if the lifecycle creates it at booking time, slot BOOKED and outbox intent only for the successful booking.

23. Failure injection and rollback evidence

INJECTION POINT	EXPECTED DATABASE STATE AFTER FAILURE	EVIDENCE TO CAPTURE
After request update, before allocation insert	Request remains PENDING; no allocation; no acceptance event.	Before/after SELECTs and transaction log.
After allocation insert, before request update	Allocation insert rolled back; request remains PENDING.	Allocation count and request status after rollback.
After booking insert, before slot update	No booking/session/outbox survives; slot stays OPEN.	Booking count, slot status, session count, outbox count.
After slot update, before outbox insert	Slot/booking change rolls back if outbox insert fails.	Slot status and booking row after rollback.
After SQL commit, Redis unavailable	Business row remains committed; outbox not marked processed until retry succeeds.	Business row, outbox status, worker retry log.
After SQL commit, Neo4j unavailable	Business row remains committed; graph projection event remains retryable.	MySQL row, event status, later graph convergence.
Worker sends notification then crashes	Event may be retried; recipient-level idempotency protects user experience.	Provider message ID / dedupe key / outbox transition.

23.1 Acceptance evidence checklist

- DDL for PK, FK, UNIQUE, CHECK constraints and relevant indexes.
- SHOW TRIGGERS output and one valid plus one invalid state transition test.
- Procedure source and CALL output for request acceptance, booking, cancellation and session outcome.
- Two-session concurrency transcript proving a single booking can win.
- Rollback evidence: no partial booking, slot update, session or outbox event.
- Worker retry evidence for Redis/Neo4j/provider outage and outbox event recovery.
- Observed InnoDB transaction isolation setting and connection-level configuration.

24. Monitoring, audit and operational safeguards

24.1 Transaction metrics

METRIC	WHAT TO MEASURE	WHY IT MATTERS
Transaction duration	p50/p95/p99 by operation type.	Long transactions hold locks and increase contention.
Lock wait time	Waits by table/index/operation.	Shows hot rows, poor lock order or missing indexes.
Deadlock count	1213 count and involved statements.	Guides lock-order and query/index improvements.
Lock timeout count	1205 count.	Indicates contention or transactions held open too long.
Duplicate conflict count	1062 by constraint name.	Distinguishes expected idempotency replay from defects.
Rollback rate	Business validation rollback vs unexpected SQL error.	Shows failed workflows and reliability.
Outbox age / backlog	Oldest pending event, pending count and retries.	Detects delivery/projection delays.
Event delivery latency	Created_at to processed_at by event type.	Measures notification and projection freshness.

24.2 Logging rules

• Log request/correlation ID, actor ID, operation, resource ID, transaction outcome and mapped error class.

• Do not log passwords, OTPs, bearer tokens, private review text or full sensitive event payloads.

• Record event_id, aggregate ID/version, worker ID, attempt number and last error for outbox tracing.

• Use audit_log for durable business change history; Redis is not an audit store.

24.3 Operational safeguards

• Set DB connection/statement timeouts and size the pool so requests do not exhaust connections.

• Monitor long-running transactions and terminate/repair leaked transaction scopes.

• Use a least-privilege database account; grant procedure/required-table access rather than unrestricted write access where practical.

• Keep external provider credentials outside source control and redact secrets from logs.

25. Decisions the project team must finalize

The supplied documents establish the core invariants, but several policy and implementation details remain intentionally configurable. They should be resolved before production-style implementation.

DECISION	SOURCE STATUS / WHY IT MATTERS	RECOMMENDED ACTION
Mentor maximum active mentees	Capacity is checked, but the exact limit is a project policy.	Store capacity in mentor profile/configuration and enforce while mentor row is locked.
Booking cancellation cutoff	Source documents say apply a cutoff but do not set a number.	Confirm cutoff duration, time zone and slot-release rule.
Can a cancelled slot be rebooked?	Workflow says reopen only when policy permits.	Define cancellation state and active_slot uniqueness strategy.
Create SCHEDULED session at booking commit?	Workflow / PRD distinguish booking and session; workflow test expects a SCHEDULED session after confirmed booking.	Adopt a single rule: create the session in the same booking transaction or create it in a separate explicit transaction/event. Prefer atomic creation if required by workflow.
Isolation level in deployment	InnoDB commonly defaults to REPEATABLE READ, but actual setting must be checked.	Record exact MySQL global/session setting; test locks under that setting.
Canonical availability status	The Business Rules Catalogue uses OPEN, while a Workflow SQL example uses AVAILABLE.	Choose one canonical persisted value; code examples in this specification use OPEN.
Lock timeout and retry limit	No fixed values are specified.	Choose bounded retry count and jitter from measured test environment.
Notification provider dedupe support	Outbox retry can repeat after crash.	Use provider idempotency key where available; retain event/recipient dedupe key.
Outbox retention and dead-letter policy	Operational details not fixed.	Set retention, alert threshold, manual re-drive procedure and payload privacy.
Admin overrides	May be allowed only through audited correction.	Define permitted override actions and reason capture.

IMPORTANT
Do not report proposed policy values as if they were requirements in the original blueprint. Label them as team-approved configuration once finalized.

26. Implementation roadmap

PHASE	IMPLEMENTATION WORK	EXIT CRITERION
1. Relational foundation	Finalize schema, FK/UNIQUE/CHECK constraints, booking indexes, request/allocation/booking/session status values.	DDL review passes; key constraints proven with negative tests.
2. Transaction services	Implement accept/reject/cancel request, availability publish, booking, cancellation, session outcome, feedback/review.	Each workflow has a single documented transaction boundary and rollback path.
3. Locking protocol	Implement canonical row-lock order, parent guard locks for overlap/capacity, conditional updates and error mapping.	Two-connection concurrency suite passes repeatedly.
4. Transactional outbox	Insert event rows within business transactions; add worker claim, lease, retry, dead-letter and dedupe.	Worker outage/retry and false-notification rollback tests pass.
5. Secondary projections	Wire Redis invalidation and Neo4j projection from committed outbox events.	Redis/Neo4j outage does not affect MySQL business correctness; retry converges.
6. Observability and demo	Add correlation IDs, transaction metrics, deadlock logs, outbox backlog monitoring and reproducible demo data.	Evidence bundle documents ACID, concurrency, rollback and retry tests.

26.1 Suggested academic demonstration sequence

1. Submit a pending mentorship request. 2. Accept it and show request + allocation + outbox all committed. 3. Run two simultaneous booking attempts for one slot. 4. Inject a failure before commit and prove complete rollback. 5. Stop the outbox worker, create a valid business event, restart the worker and show retry. 6. Demonstrate that Redis and Neo4j can be unavailable without allowing an invalid booking or rolling back a committed business transaction.

27. Traceability to previously supplied project documents

SOURCE DOCUMENT	RELEVANT CONTENT USED IN THIS SPECIFICATION
Alumni–Student Mentorship Platform: DBMS Project Blueprint	Modular monolith; MySQL system of record; Redis performance/temporary state; Neo4j recommendation projection; request/allocation/booking/session distinctions; outbox principle.
AlumniConnect Application Architecture	Section 8 / p. 11: transactional outbox and mentor skill update sequence. Sections 9 / pp. 12–13: booking concurrency and locking/transaction responsibilities.
AlumniConnect Workflow and State Diagrams	Event delivery / p. 11; database automation / pp. 12–13; procedures and constraints / pp. 18–20; Unit 4 transaction catalogue / p. 21.
AlumniConnect Business Rules Catalogue	Section 6 / pp. 9–10: availability, booking and half-open overlap rules. Section 12 / pp. 24–25: ACID, anomalies, locks, isolation and deadlocks. Verification catalogue / pp. 28–29.
AlumniConnect Product Requirements Document	FR-20 and FR-23 / p. 7: outbox and Redis management. Business rules and source-of-truth boundary / pp. 8–10.
AlumniConnect Conceptual ER Model	Entity and relationship separation for request, allocation, availability, booking, session, feedback, review, outbox and audit.
AlumniConnect Use Case Model	Actor permissions and role-specific actions for student, mentor and administrator.
AlumniConnect Redis Design Specification	Cache invalidation after commit; outbox event mapping; Redis idempotency and temporary-state limitations; worker retry safety (Sections 9–12, pp. 11–14).
AlumniConnect Neo4j Graph Design	Neo4j is a recommendation projection; graph recommendations are candidates and MySQL validates current eligibility.

27.1 Source-derived versus recommended details

The ACID guarantees, one-transaction business boundaries, row locks, fixed lock order, MySQL source-of-truth rule, outbox requirement and concurrency tests are directly supported by the supplied documents. The exact retry counts, backoff parameters, worker lease durations, outbox retention, numeric timeouts and cancellation cutoff are not fixed in those sources; this document presents them as configurable implementation decisions.

Appendix A. Practical SQL patterns

A.1 Check transaction isolation

```
SELECT @@transaction_isolation;
SELECT @@global.transaction_isolation;
```

A.2 Confirm slot still open under lock

```
SELECT availability_id, mentor_id, status, start_at, end_at
FROM mentor_availability
WHERE availability_id = :slot_id
FOR UPDATE;

-- Validate status = 'OPEN', mentor matches, start_at > NOW().
```

A.3 Conditional state update

```
UPDATE mentor_availability
SET status = 'BOOKED'
WHERE availability_id = :slot_id
AND status = 'OPEN';

IF ROW_COUNT() <> 1 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Slot is no longer available';
END IF;
```

A.4 Find conflicting appointments

```
SELECT booking_id
FROM booking
WHERE booking_status = 'CONFIRMED'
AND start_at < :requested_end
AND end_at > :requested_start
AND (student_id = :student_id OR mentor_id = :mentor_id)
LIMIT 1;
```

Run the overlap check only after locking the relevant student and mentor guard rows. Include the chosen final table aliases and indexes after schema freeze.

Appendix B. Review checklist for every state-changing operation

CHECK	QUESTION TO ANSWER	PASS CONDITION
Authorization	Is the actor authenticated and authorized for this exact resource?	Backend verifies role and ownership on every call.
Transaction boundary	Are all required durable writes in one transaction?	Commit or rollback covers every authoritative write.
Locks	Are all decision rows locked in a consistent order?	All competing writers use the same protocol.
Re-read	Is state re-validated after acquiring locks?	Pre-lock data is not trusted as current authority.
Constraints	Which DB constraint is the final backstop?	Unique/FK/check rules reject illegal state.
Outbox	Are side-effect intents recorded inside the transaction?	Rollback leaves no deliverable event.
No remote calls	Are Redis/Neo4j/provider calls outside lock-holding transactions?	No network wait while business locks are held.
Idempotency	What happens on a client retry or worker retry?	Same durable result; no duplicate business effect.
Failure mapping	Are deadlock, timeout, duplicate and business errors distinguished?	Safe retry or meaningful validation response.
Audit	Can a reviewer determine actor, record, state change and timestamp?	History is durable in MySQL.
Tests	Are race, rollback and worker-failure tests included?	Independent-connection tests pass and evidence is captured.

B.1 Minimum acceptance statement

TRANSACTIONAL CORRECTNESS
For every critical AlumniConnect workflow, the system commits one valid business transition and its durable event intent atomically; prevents concurrent transactions from violating uniqueness, capacity or overlap rules; and tolerates secondary-store or notification failures without corrupting MySQL state.

Appendix C. Key terms

TERM	MEANING IN THIS DESIGN
ACID	Atomicity, Consistency, Isolation and Durability - transaction properties.
Aggregate / guard row	A stable parent row locked to serialize operations that affect a shared capacity or overlap rule.
Deadlock	A cycle of transactions waiting for one another’s locks; InnoDB rolls back a victim.
Idempotency key	A client-supplied unique token used to recognize a retry of the same logical request.
InnoDB row lock	A lock acquired by a transaction on selected records/index records to coordinate concurrent work.
Outbox	A durable event row inserted in the same transaction as the business change.
Phantom / overlap race	A concurrent insert creates a row matching a predicate that another transaction previously checked as empty.
Projection	A derived, rebuildable representation in Redis or Neo4j that is not authoritative business state.
Serialization point	A row/resource whose lock forces competing operations to take turns for a shared invariant.
Two-phase locking	A locking protocol in which locks are acquired before release; helps provide serializable conflict behaviour but does not eliminate deadlocks.

Final design principle

The strongest version of AlumniConnect is not the version that puts every action into Redis, Neo4j or a separate service. It is the version that gives each database a clear responsibility, defines exactly where each business transaction begins and ends, protects its invariants under concurrency, and makes every secondary effect durable and retryable.

> FINAL RULE
> MySQL commits the business truth. The outbox records what must happen next. Workers deliver and project after commit. Redis and Neo4j accelerate discovery, but never override the committed relational state.